

# ExpenseSense: Privacy-Preserving On-Device Personal Finance Tool Calling with Sub-2B LLMs

Anonymous ACL submission

## Abstract

For personal finance, cloud-based inference poses privacy risks, highlighting the need for local on-device execution. Yet, the tool-calling performance of sub-2B small language models (SLMs) remains under-explored. We introduce **ExpenseSense**, a privacy-first personal finance assistant, and benchmark on-device tool calling using a 115-question test suite. We compare single-agent (joint routing and parameter extraction) against dual-agent (router-specialist) pipelines across six local models (0.8B–2B). Dual-agent decomposition improves correctness (e.g., Qwen3.5-2B Correct Ratio improves from 60.2% to 77.6%), as well as decreases average latency by up to 48% (e.g., from 12.0 s to 6.2 s for Qwen3.5-2B). It also shifts errors toward safer failure modes. Qwen3.5-0.8B dual achieves 89.1% Task Accuracy with 3.0 s latency and 1.2 GB RAM, demonstrating the viability of fully private, on-device financial assistants. All code, data, and prompts are available on Github.

## 1 Introduction

Large language models (LLMs) have rapidly advanced tool-augmented AI (Schick et al., 2023; Patil et al., 2024; Qin et al., 2024), enabling agents to invoke structured APIs from natural language. However, research primarily targets large cloud-hosted models, which is problematic for personal finance where local inference is crucial for privacy. Expense records reveal sensitive behavioral patterns, yet users expect natural language interfaces (e.g., asking “how much did I spend on food last month?”) that map to structured queries over local data (Li et al., 2024). This creates an underexplored problem: can *sub-2B quantized models* running entirely on consumer hardware reliably call domain-specific financial tools from informal, colloquial language?

Existing benchmarks (Patil et al., 2025; Qin et al., 2024; Zhu et al., 2026; Cao et al., 2026) fo-

cus on models above 7B parameters, which are prohibitively slow for local deployment. Conversely, on-device models like Octopus v2 (Chen and Li, 2024) and Hammer (Lin et al., 2024) demonstrate the potential of smaller models but have not been evaluated on domain-specific financial schemas or under consumer-hardware latency constraints.

We address three research questions (RQ):

- **RQ1:** Can sub-2B quantized SLMs achieve reliable personal finance analytics tool calling on consumer hardware, enabling fully private local inference?
- **RQ2:** Does decomposing intent classification from parameter extraction (dual-agent) yield significant accuracy gains over a joint single-agent architecture?
- **RQ3:** What are the primary failure modes across architectures, and how do they differ qualitatively?

This paper introduces **ExpenseSense**, with the following contributions:

1. A **115-question dataset** for financial tool calling over five analytical tools, with a 7-feature complexity scoring function producing three difficulty tiers (L1-L3), designed to simulate real-world human interactions with financial assistants.
2. A comparative evaluation of **single-agent vs. dual-agent** architectures across six quantized SLMs (0.8B-2B parameters), demonstrating that while improved single-agent prompts can reach high Task Accuracy (92.0%), dual-agent decomposition is critical for faster inference (e.g., up to 48% latency reduction for Qwen3.5-2B), strict exact-match correctness (+17.4 pp Correct Ratio) and for stabilizing smaller sub-1B models without cloud dependence.

3. A **fine-grained error taxonomy** (7 error types) revealing qualitatively different failure modes between architectures—dual-agent consolidates errors into more predictable categories, a desirable property for safety-critical financial applications.

4. Practical **on-device deployment guidance** for human-centered privacy: Qwen3.5-0.8B in dual mode achieves 89.1% Task Accuracy at 3.0 s latency and  $\approx 1.2$  GB RAM, demonstrating that users can access high-quality personal finance analytics on consumer hardware.

## 2 Privacy Motivation

The deployment of LLMs in personal finance represents a typical case of the tension between AI capability and human privacy. Cloud-based LLM services—whether general-purpose assistants or specialised financial tools—require users to transmit queries containing detailed spending information to external servers. Even when the raw financial database is not uploaded, the *queries themselves* reveal sensitive information: a question like “how much did I spend on medical expenses last month?” discloses both health-related spending and temporal patterns. [Staab et al. \(2024\)](#) demonstrated that LLMs can infer personal attributes (such as income, location, and health conditions) from seemingly innocuous text ([Staab et al., 2024](#)), and [Yao et al. \(2024\)](#) catalogue the broad landscape of LLM privacy risks, including data leakage, memorization, and inference attacks.

ExpenseSense addresses this threat by adopting an *on-device, privacy-first* architecture. By leveraging sub-2B quantized models that run efficiently on consumer hardware (tested on Apple M1 with 8 GB unified memory), all user data and queries remain strictly local. No financial records, no query text, and no model outputs ever leave the device. This design aligns with [Nissenbaum’s](#) framework of *contextual integrity*: financial data generated within the context of personal expense tracking should remain within that context, not flow to third-party model providers whose informational norms may differ from the user’s expectations.

The key challenge is that on-device inference with sub-2B models has historically been unreliable for structured tool calling ([Kavathekar et al., 2025](#)). Our work demonstrates that architectural decomposition—separating intent classification from parameter extraction—bridges this reliability gap, making on-device private inference a practical reality rather than a theoretical aspiration.

## 3 Related Work

**Tool-Augmented Language Models.** Toolformer ([Schick et al., 2023](#)) demonstrated self-supervised tool-use learning; Gorilla ([Patil et al., 2024](#)) and ToolLLM ([Qin et al., 2024](#)) scaled to thousands of real-world APIs. HuggingGPT ([Shen et al., 2023](#)) and ToolAlpaca ([Tang et al., 2023](#)) explored multi-step orchestration. APIGen ([Liu et al., 2024b](#)) introduced a verified pipeline for generating function-calling training data, showing that even 1B-parameter models fine-tuned on high-quality synthetic data can surpass GPT-3.5-Turbo on structured tool invocation. All of these works target models  $\geq 7$ B with cloud inference, leaving the privacy-constrained on-device setting unexplored.

**Function-Calling Evaluation.** The Berkeley Function Calling Leaderboard ([Patil et al., 2025](#)) established a comprehensive benchmark using AST substring matching and execution-based validation. [Kavathekar et al. \(2025\)](#) proposed an evaluation suite—FSP, ACS, AVC, Task Accuracy, and Correct Ratio—specifically designed for SLM function call generation, operating on JSON dictionaries without code execution; we adopt these metrics directly. CallNavi ([Song et al., 2025](#)) proposed a two-stage router+specialist approach—the closest architectural analogue to our dual-agent design—but evaluates models  $\geq 7$ B on general-purpose APIs. We study the orthogonal case of sub-2B models on a domain-specific, single-turn financial schema under strict privacy constraints.

**On-Device Function Calling.** MobileLLM ([Liu et al., 2024a](#)), Phi-3 ([Abdin et al., 2024](#)), and related surveys ([Xu et al., 2024](#)) have explored sub-4B parameter models for mobile deployment. Octopus v2 ([Chen and Li, 2024](#)) fine-tuned a 2B model for on-device API calling via functional tokens, achieving competitive latency against GPT-4 on specific tasks. Hammer ([Lin et al., 2024](#)) introduced function masking to reduce overfitting to function names, a problem we observe indirectly through our category normalisation errors. Apple Intelligence ([Gunter et al., 2024](#)) validated on-device tool calling at scale with a  $\sim 3$ B model, demonstrating that the paradigm is production-ready. [Kavathekar et al. \(2025\)](#) found that struc-

| Tool                 | ID | Key Params                                                   |
|----------------------|----|--------------------------------------------------------------|
| plot_time_series     | 1  | category, year/month, ranges, months, ignore_rent            |
| plot_distribution    | 2  | category, year/month/day, ranges, months, ignore_rent        |
| plot_comparison_bars | 3  | category, y1/m1, y2/m2, sm/em/ey, ignore_rent                |
| calculate_total      | 4  | category, remarks, year/month/day, months, ignore_rent       |
| get_top_expenses     | 5  | n, category, year/month/day, months, min_amount, ignore_rent |

Table 1: Tool inventory. The five tools span visualisation (1–3), aggregation (4), and retrieval (5).

includes structured output rules, date-handling instructions, and 7 tool-specific examples aggregated across all 5 tools. Prompt length:  $\approx 5,500$  tokens (context window inclusive).

**Dual-Agent Pipeline.** The task is decomposed into two sequential inference calls:

1. **Router:** A focused intent-classification prompt with 19 few-shot examples and explicit disambiguation rules (e.g., “compare” requires two distinct time periods) elicits a single digit (1–5).
2. **Specialist:** The predicted tool ID selects a tool-specific prompt containing 12–15 examples for that *one* tool, along with detailed parameter definitions and date-handling rules.

The router sees  $\approx 1,000$  tokens; the specialist sees  $\approx 1,500$ – $2,000$  tokens with deeper, more directly relevant supervision. This reduces per-call cognitive load and provides richer in-context learning per subtask, at the cost of two sequential inference passes. Combined prompt tokens:  $\approx 2,700$  per query.

### 4.3 Post-Hoc Parameter Validation

After each inference call, a deterministic `validate_and_fix_params` layer corrects category strings without additional model calls:

1. **Exact match** against the known vocabulary.
2. **Morphological normalisation:** `groceries`  $\rightarrow$  `grocery`, `cafés`  $\rightarrow$  `cafe`.
3. **Edit-distance matching** via `diff`lib (cutoff  $\geq 0.80$ ).
4. **Token overlap scoring** (Jaccard; cutoff  $\geq 0.50$ ).

All results report both raw (pre-validation) and validated (post-validation) metrics.

## 5 Benchmark Design

### 5.1 Test Suite

The benchmark comprises 115 test cases across five task groups (Table 2). Queries are written in naturalistic, informal language: abbreviations (“*compare electricity* ’24 vs ’25”), colloquialisms (“*can ya tell me*”), misspellings (“*distribuuution*”), and domain-specific terms (“*combinis*”, “*nomikais*”,

tured output compliance is the primary bottleneck for SLMs—a finding that directly motivates our architectural decomposition approach.

**Financial NLP and Privacy.** FinQA (Chen et al., 2021) and FinGPT (Yang et al., 2023) target financial numerical reasoning and generation, but do not address tool-calling or privacy constraints. FinMCP-Bench (Zhu et al., 2026) and FinTrace (Cao et al., 2026) evaluate LLM agents for financial tool use, but target cloud-scale models. Personal finance assistants (Li et al., 2024) represent an emerging application domain where on-device inference is especially valuable, as users may prefer to keep financial data local for privacy or lack affordable access to cloud-hosted LLMs.

## 4 System Design

### 4.1 Tool Inventory

ExpenseSense exposes five analytical tools over a user’s expense dataframe (columns: date, amount, major category, subcategory, remarks). Table 1 summarises each tool and its parameter space. The tools cover core personal finance use cases: time series analysis, proportional breakdown, period comparison, sum aggregation, and ranked retrieval.

### 4.2 Architectural Configurations

We evaluate two distinct configurations for mapping natural language queries to tool calls, both designed for fully local, private execution.

**Single-Agent Pipeline.** One model receives a comprehensive prompt containing all five tool definitions, parameter schemas, and cross-tool few-shot examples. It must jointly determine the correct tool *and* extract all parameters in one inference pass, outputting a JSON object with a `tool` field (1–5) and corresponding parameters. The prompt

| Task Group      | Cases      | Avg Params | Avg $C$    |
|-----------------|------------|------------|------------|
| Time Series     | 23         | 2.9        | 7.0        |
| Distribution    | 21         | 2.5        | 6.4        |
| Comparison      | 28         | 4.4        | 8.8        |
| Calculate Total | 22         | 2.4        | 6.3        |
| Top Expenses    | 21         | 3.2        | 6.7        |
| <b>Total</b>    | <b>115</b> | <b>3.2</b> | <b>7.1</b> |

Table 2: Test case distribution by group. Comparison queries are the most parameter-dense and complex.

| Model           | Family            | Params | RAM    |
|-----------------|-------------------|--------|--------|
| EXAONE-4.0-1.2B | EXAONE-4.0 (2025) | 1.2B   | 2.0 GB |
| Gemma-3-1B-it   | Gemma-3 (2025)    | 1B     | 1.6 GB |
| LFM2.5-1.2B     | LFM2.5 (2026)     | 1.2B   | 1.4 GB |
| MiniCPM5-1B     | MiniCPM (2024)    | 1B     | 1.5 GB |
| Qwen3.5-0.8B    | Qwen3.5 (2026)    | 0.8B   | 1.3 GB |
| Qwen3.5-2B      | Qwen3.5 (2026)    | 2B     | 2.1 GB |

Table 3: Models evaluated. All use Q8\_0 GGUF quantization via llama.cpp. RAM is average peak RSS.

258 “*shinkansen*”). This reflects the Japan-based per-  
259 sonal finance context of the underlying system  
260 and tests robustness to real-world query diversity—  
261 the kind of informal, spontaneous interactions that  
262 characterise actual use of personal financial assis-  
263 tants.

## 264 5.2 Complexity Scoring

265 Each test case is assigned a difficulty score:

$$266 \quad C = n_p + d_{\text{date}} + d_{\text{cat}} + d_{\text{rel}} + d_{\text{multi}} + d_{\text{abbr}} + d_{\text{hol}} \quad (1)$$

267 where  $n_p$  = number of expected parameters;  
268  $d_{\text{date}} \in \{0-4\}$  scores date specification complex-  
269 ity;  $d_{\text{cat}} \in \{0-2\}$  is the category normalisation  
270 distance;  $d_{\text{rel}} \in \{0, 1\}$  flags relative time expres-  
271 sions;  $d_{\text{multi}} \in \{0-2\}$  counts compound parame-  
272 ter groups;  $d_{\text{abbr}} \in \{0, 1\}$  flags abbreviated dates;  
273 and  $d_{\text{hol}} \in \{0, 2\}$  flags calendar knowledge re-  
274 quirements. Scores are binned into three tiers: **L1**  
275 ( $C \leq 4$ , 24 cases), **L2** ( $5 \leq C \leq 7$ , 45 cases), **L3**  
276 ( $C \geq 8$ , 46 cases).

## 277 5.3 Evaluation Metrics

278 We adopt the metric suite of Kavathekar et al.  
279 (2025):

- 280 • **FSP** (Function Selection Precision): Binary  
281 indicator of correct tool prediction.
- 282 • **ACS** (Argument Coverage Score):  $F_1$  on ar-  
283 gument *name* overlap.
- 284 • **AVC** (Argument Value Correctness): Fraction  
285 of correctly named arguments whose values  
286 also match.
- 287 • **Task Accuracy**:  $F_1$  on the combined set  
288  $\{\text{tool}\} \cup \{k=v\}$ , jointly capturing tool selec-  
289 tion and parameter correctness.
- 290 • **Correct Ratio (CR)**: Strict binary indicator—  
291 1.0 only when Task Acc = 1.0, i.e. the entire  
292 function call is perfectly correct.

293 All metrics are averaged across 5 repetitions per  
294 test case (temperature = 0.1) with 95% bootstrap  
295 confidence intervals (Efron, 1979) ( $n = 2000$  re-  
296 samples).

## 297 5.4 Error Taxonomy

298 Each prediction is classified into one  
299 of seven mutually exclusive error types:  
300 CORRECT, TOOL\_WRONG, PARSE\_FAILURE,  
301 PARAM\_MISSING, PARAM\_VALUE\_WRONG,  
302 PARAM\_EXTRA, FORMAT\_VIOLATION. This  
303 enables targeted analysis of *what* fails rather than  
304 just *how much*, which is essential for assessing the  
305 safety profile of on-device financial assistants.

## 306 6 Experimental Setup

### 307 6.1 Models

308 All six models (Table 3) use Q8\_0 GGUF quantiza-  
309 tion and run via llama.cpp (Gerganov, 2023) with  
310 full GPU offload on Apple Metal.

### 311 6.2 Hardware and Inference Protocol

312 Experiments run on an Apple M1 (8-core CPU, 7-  
313 core GPU, 8 GB unified memory)—a consumer  
314 laptop representative of privacy-constrained de-  
315 ployment environments. Parameters: temperature  
316 = 0.1, min\_p = 0.05, n\_ctx = 4096, stop to-  
317 kens  $[\text{"\\n\\n"}, \text{"\\n"}]$ . Each model is warmed  
318 up through all six prompt templates (one router  
319 + five specialists) before evaluation begins. Each  
320 of the 115 test cases is run 5 times per model per  
321 architecture, yielding  $115 \times 5 \times 6 \times 2 = 6,900$  total  
322 observations.

## 7 Results

### 7.1 Overall Performance

Table 4 presents the full results. Figure 1 shows the accuracy–latency Pareto frontier and the error taxonomy breakdown.

**Key findings (RQ1, RQ2).** (1) Sub-2B SLMs can reliably call financial tools on consumer hardware (RQ1: yes), with the Qwen3.5 family achieving >89% Task Accuracy in dual mode. This eliminates the need for cloud inference and keeps all user financial data strictly local. (2) While improved prompt engineering has substantially raised single-agent performance (Qwen3.5-2B reaches 92.0% Task Accuracy), dual-agent decomposition remains critical for two reasons (RQ2): (a) Strict correctness—Correct Ratio improves from 60.2% to 77.6% (+17.4 pp) for Qwen3.5-2B, meaning more queries out of every 100 are *perfectly* correct with no parameter errors. For financial applications where a single wrong parameter can produce misleading analytics, this improvement is substantial. (b) Small model enablement—Qwen3.5-0.8B improves from 79.8% to 89.1% Task Accuracy (+9.3 pp) and *halves* its latency (5.8 s→3.0 s) in dual mode, making sub-1B models viable for memory-constrained devices. (3) MiniCPM5-1B emerges as a strong mid-tier performer, reaching 74.3% Task Accuracy in dual mode with 43.5% Correct Ratio—a notable result for a 1B model.

### 7.2 Per-Tool-Group Analysis

Table 5 shows per-group accuracy for the top two dual-mode models. Figure 2 shows the full model × category heatmap. `get_top_expenses` is nearly solved (98.9% for Qwen3.5-2B)—its parameter space is simple and unambiguous. `time_series` is the most challenging (74.4% for Qwen3.5-2B), owing to diverse date range specifications and relative time expressions (“past X months”) that require temporal reasoning.

### 7.3 Complexity Tier Analysis

Figure 3 presents the per-model accuracy breakdown across complexity tiers in dual mode. Qwen3.5-2B degrades gracefully from 96% (L1) to 82% (L3), while Qwen3.5-0.8B shows a similar trajectory (95%→84%). L3 queries involve constructs like cross-year range comparisons (“compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026”) requiring up to 10 parameters with interdependent temporal semantics.

### 7.4 Error Taxonomy Analysis

Figure 4 visualises the full error distribution. The shift in error profiles answers RQ3 directly.

For Qwen3.5-2B: single-agent produces 22.6% `PARAM_EXTRA` and 7.3% `PARAM_MISSING`; dual-agent reduces `PARAM_EXTRA` to 1.9% while `PARAM_MISSING` remains at 8.0%. The elimination of `PARAM_EXTRA` errors is particularly important for financial safety: a spurious `months=1` filter added to “how much did I spend on food?” silently restricts the query to the last month, producing a misleadingly low total. Dual-agent’s residual `PARAM_MISSING` errors are a *safer* failure mode—the system omits a filter, producing a broader result, which is less likely to cause financial misjudgements.

For Qwen3.5-0.8B: single-agent produces 32.5% `PARAM_MISSING` and 11.8% `PARAM_EXTRA`; dual-agent reduces these to 12.9% and 11.1% respectively, shifting the model from a high-variance to a more predictable failure profile.

### 7.5 Token Efficiency

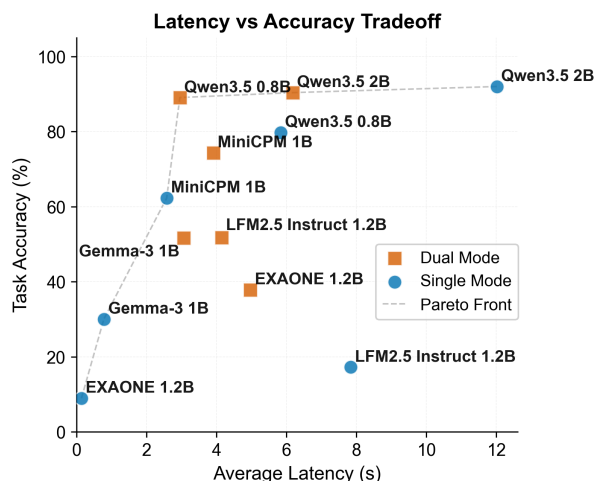
Figure 5 shows that despite dual-agent using ≈2,700 prompt tokens per query (vs. ≈5,500 for single-agent due to the comprehensive multi-tool prompt), it achieves equivalent or superior accuracy. The single-agent pipeline processes *more* context but converts it less efficiently into correct tool calls, because the model must simultaneously attend to five tool schemas and cross-tool disambiguation. The dual-agent architecture processes less context per call but achieves higher accuracy-per-token by focusing each inference pass on a single subtask.

### 7.6 Validation Layer Impact

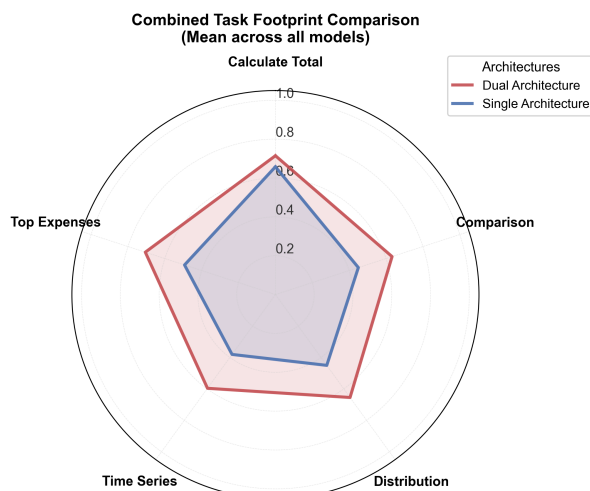
The validation layer (Table 6) contributes modest gains, primarily correcting category casing/normalisation. Its limited impact is *informative*: since `PARAM_MISSING` is the dominant residual failure mode and validation cannot recover missing parameters, the primary bottleneck is structural extraction, not surface normalisation.

## 8 Discussion

**Privacy through Task Decomposition.** The dual-agent architecture serves a dual purpose: it improves accuracy *and* makes privacy-preserving local inference viable. By separating routing (single-digit output) from extraction (focused JSON), dual-agent reduces the structured output burden per call,



(a) Accuracy–latency Pareto frontier. The dual-mode Qwen3.5-0.8B (orange square,  $\approx 3$  s, 89%) sits on the efficient frontier, offering the best accuracy per unit latency. Single-mode Qwen3.5-2B reaches the highest Task Accuracy (92%) but at  $4\times$  the latency (12 s).



(b) Task footprint (mean Task Accuracy across all models). The dual architecture (red) uniformly expands coverage over single (blue) on all five task groups, with the largest gains on time series and comparison tools.

Figure 1: System-level results. (a) Dual-agent Qwen3.5-0.8B dominates the efficiency frontier for privacy-preserving deployments. (b) Dual-agent decomposition improves coverage across all task categories.

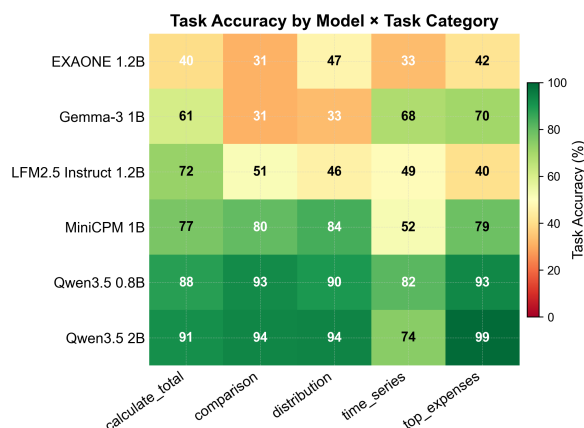


Figure 2: Task Accuracy (%) by Model  $\times$  Tool Category in dual mode. Qwen3.5 models achieve  $>74\%$  across all categories, while weaker models show pronounced gaps in comparison and time series tasks.

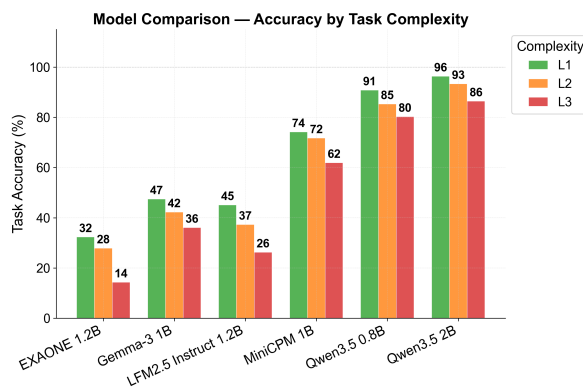


Figure 3: Task Accuracy by complexity tier (L1/L2/L3) per model in dual mode. The Qwen3.5 family degrades gracefully with increasing complexity.

420 which is precisely the bottleneck that Kavathekar et al. (2025) identify as the primary limitation of SLMs for function calling. This cognitive flooding is what makes sub-2B models competitive with much larger cloud models for domain-specific tasks, enabling fully private local execution.

426 **Why dual-agent still matters despite strong single-agent prompts.** A natural question is whether improved prompt engineering—which has raised single-agent Qwen3.5-2B to 92.0% Task Accuracy—has rendered the dual-agent architecture unnecessary. Our results argue strongly against this: (1) The 17.4 pp gap in Correct Ratio (60.2%  $\rightarrow$  77.6%) means that single-agent produces wrong parameters in 40% of queries vs. 22% for dual-agent. (2) For the privacy-optimal Qwen3.5-0.8B (which fits in  $<1.5$  GB RAM), dual-agent provides a +9.3 pp Task Accuracy boost and halves latency, because shorter, more focused prompts generate faster on small models. (3) The error taxonomy shift from PARAM\_EXTRA (dangerous in finance) to PARAM\_MISSING (safer) represents a qualitative safety improvement.

429 **The EXAONE anomaly.** EXAONE-4.0-1.2B achieves 90.6% FSP in dual mode—excellent routing—but averages near-empty specialist out-

| Mode   | Model             | FSP         | ACS  | AVC         | Task Acc    | CR          | Latency (s) | RAM (GB) |
|--------|-------------------|-------------|------|-------------|-------------|-------------|-------------|----------|
| Single | EXAONE-4.0-1.2B   | 19.3        | —    | 0.3         | 8.9         | 0.3         | 0.1         | 2.0      |
|        | Gemma-3-1B-it     | 27.0        | —    | 47.5        | 30.0        | 0.9         | 0.8         | 1.6      |
|        | LFM2.5-1.2B       | 28.7        | —    | 11.7        | 17.2        | 3.1         | 7.8         | 1.4      |
|        | MiniCPM5-1B       | 71.1        | —    | 86.7        | 62.3        | 13.0        | 2.6         | 1.5      |
|        | Qwen3.5-0.8B      | 95.0        | —    | 91.2        | 79.8        | 40.2        | 5.8         | 1.3      |
|        | <b>Qwen3.5-2B</b> | <b>96.5</b> | —    | <b>97.6</b> | <b>92.0</b> | 60.2        | 12.0        | 2.1      |
| Dual   | EXAONE-4.0-1.2B   | 90.6        | 3.7  | 3.3         | 37.8        | 3.0         | 5.0         | 1.9      |
|        | Gemma-3-1B-it     | 55.1        | 59.9 | 71.6        | 51.7        | 16.5        | 3.1         | 1.5      |
|        | LFM2.5-1.2B       | 69.6        | 51.6 | 57.2        | 51.8        | 16.3        | 4.1         | 1.4      |
|        | MiniCPM5-1B       | 84.9        | 77.0 | 81.6        | 74.3        | 43.5        | 3.9         | 1.5      |
|        | Qwen3.5-0.8B      | <b>94.3</b> | 90.2 | 96.8        | 89.1        | 61.7        | 3.0         | 1.2      |
|        | <b>Qwen3.5-2B</b> | 92.0        | 91.6 | <b>97.4</b> | 90.4        | <b>77.6</b> | 6.2         | 2.1      |

Table 4: Main results (validated metrics, %). **Bold** = best per metric. FSP = Function Selection Precision; ACS = Argument Coverage Score; AVC = Argument Value Correctness; Task Acc =  $F_1$ -based task accuracy; CR = strict exact-match Correct Ratio. ACS is omitted (—) where the single-agent architecture’s joint output makes isolated argument coverage less meaningful. Latency and RAM are per-query averages.

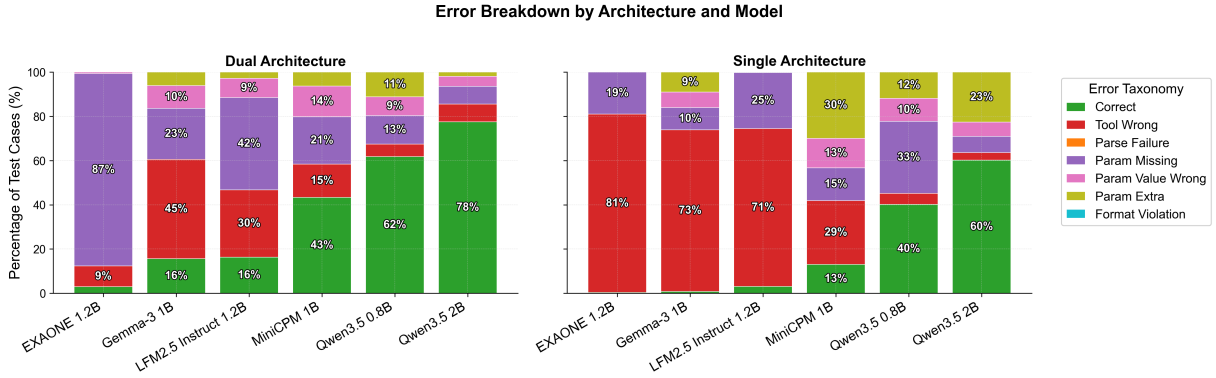


Figure 4: Error taxonomy breakdown (% of test cases) per model and architecture. Single-agent Qwen3.5-2B (right panel) produces 22.6% PARAM\_EXTRA errors—spurious parameters that could lead to incorrect financial calculations. Dual-agent (left) eliminates most of these, consolidating residual errors into the more predictable PARAM\_MISSING category.

| Group        | Qwen3.5-0.8B |      | Qwen3.5-2B |      |
|--------------|--------------|------|------------|------|
|              | T.Acc        | CR   | T.Acc      | CR   |
| Time Series  | 81.6         | 54.8 | 74.4       | 60.9 |
| Distribution | 89.6         | 56.2 | 93.8       | 79.0 |
| Comparison   | 92.5         | 71.4 | 93.7       | 73.6 |
| Calc. Total  | 87.9         | 59.1 | 91.4       | 84.5 |
| Top Expenses | 93.3         | 64.8 | 98.9       | 92.4 |

Table 5: Per-group results in dual mode (validated, %). T.Acc = Task Accuracy, CR = Correct Ratio.

| Model (Dual)  | $\Delta$ Task Acc | $\Delta$ CR |
|---------------|-------------------|-------------|
| EXAONE-4.0    | +0.00             | +0.00       |
| Gemma-3-1B-it | +0.12             | +0.00       |
| LFM2.5-1.2B   | +0.50             | +0.00       |
| MiniCPM5-1B   | +1.17             | +0.87       |
| Qwen3.5-0.8B  | +0.41             | +1.57       |
| Qwen3.5-2B    | +0.29             | +0.87       |

Table 6: Validation layer impact ( $\Delta$  in percentage points, dual mode). Gains are modest but consistent for capable models.

puts (ACS = 3.7%). This strongly suggests a chat-**L3 complexity remains a hard frontier.** Even template or instruction-following constraint in the best model achieves only 59.6% strict Correct GGUF conversion that truncates structured out- Ratio on L3 queries in dual mode, which involve put, rather than a semantic failure. The function cross-year range comparisons requiring up to 10 masking approach of Lin et al. (2024) or the interdependent parameters. Fine-tuning on domain- specific examples using verified synthetic data (Liu offer paths to resolving such failures via targeted et al., 2024b) represents a promising avenue for closing this gap while preserving the privacy bene-

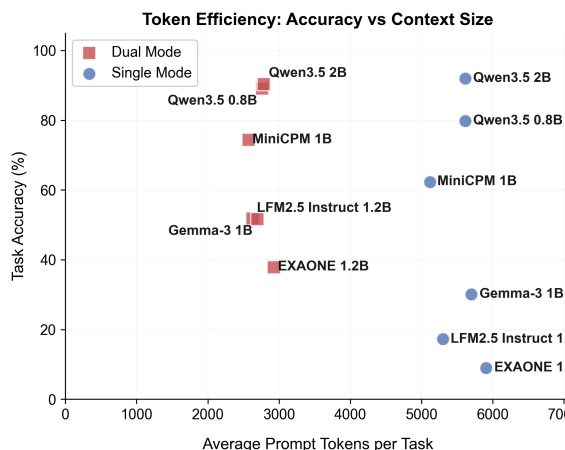


Figure 5: Token efficiency: Task Accuracy vs. average prompt tokens per query. Dual-agent configurations (red) achieve higher accuracy with fewer total tokens than their single-agent counterparts (blue), demonstrating superior context utilisation.

fits of local inference.

**Deployment recommendations for privacy-first systems.** For practitioners building privacy-preserving financial assistants: (1) *Memory constrained* ( $\leq 2$  GB available): Qwen3.5-0.8B dual, 89.1% Task Acc, 3.0 s, 1.2 GB RAM. (2) *Accuracy-first*: Qwen3.5-2B dual, 90.4% Task Acc, 77.6% CR, 6.2 s, 2.1 GB RAM. (3) *Budget hardware*: MiniCPM5-1B dual, 74.3% Task Acc, 43.5% CR, 3.9 s, 1.5 GB RAM. All configurations keep user data entirely on-device.

**Illustrative Case Study.** Figure 6 shows a real usage session of ExpenseSense. A user queried the system in informal language—“can ya plot spending on snacks for past 14 months?”—which the dual-agent Qwen3.5-0.8B pipeline correctly routed to `plot_time_series` and extracted the parameters `{category: snacks, months: 14}`. The resulting visualization revealed a clear declining trend in weekly snack expenditure (from peaks of ¥3,000–4,000 to under ¥1,000), which the user reported influenced their subsequent spending behaviour. While anecdotal, this example illustrates two properties of the system: (1) fully private on-device inference successfully handles naturalistic queries over real personal financial data, and (2) the resulting analytics are actionable—precisely the value proposition that motivates keeping financial AI local rather than cloud-dependent.

## 9 Conclusion

We presented ExpenseSense, a human-centered case study evaluating privacy-preserving on-device financial tool calling with sub-2B quantized models. Our central finding is that *architectural decomposition*—separating intent classification from parameter extraction into a router–specialist pipeline—remains critical even as single-agent prompts improve: it unlocks strict correctness (77.6% Correct Ratio vs. 60.2%) and enables sub-1B models to reach 89.1% Task Accuracy at 3.0 s, a practical operating point where no user data ever leaves the device. Error taxonomy analysis reveals that dual-agent decomposition not only improves accuracy but produces *safer* failure modes: residual errors shift from dangerous parameter additions to more benign parameter omissions, a qualitative property desirable for financial safety.

Several directions warrant future investigation. First, *fine-tuning* the best-performing models on domain-specific function-calling data could address the residual L3 failures while preserving on-device deployment. Second, extending the benchmark to *multi-turn* interactions would capture realistic conversational patterns. Third, *multilingual* query support would broaden applicability across diverse user populations. Finally, evaluating lower-bit quantisation formats (Q4\_K\_M) could further reduce the memory footprint, making privacy-preserving financial AI accessible on even more constrained devices such as smartphones.

## 522 Limitations

- The benchmark targets a single expense tracking application; generalisability to other financial domains (trading, compliance, auditing) requires further investigation.
- All models use Q8\_0 quantisation; lower-bit formats (Q4\_K\_M) may yield different accuracy–efficiency tradeoffs.
- Queries are in English and reflect a Japan-based personal finance context (categories such as “combinis”, “nomikais”, “shinkansen”). Multilingual and cross-cultural diversity is not covered.
- Ground-truth labels are deterministic; some queries may admit legitimate alternative tool selections.
- Test cases were authored by a single developer; crowdsourcing or user study collection would improve query diversity.

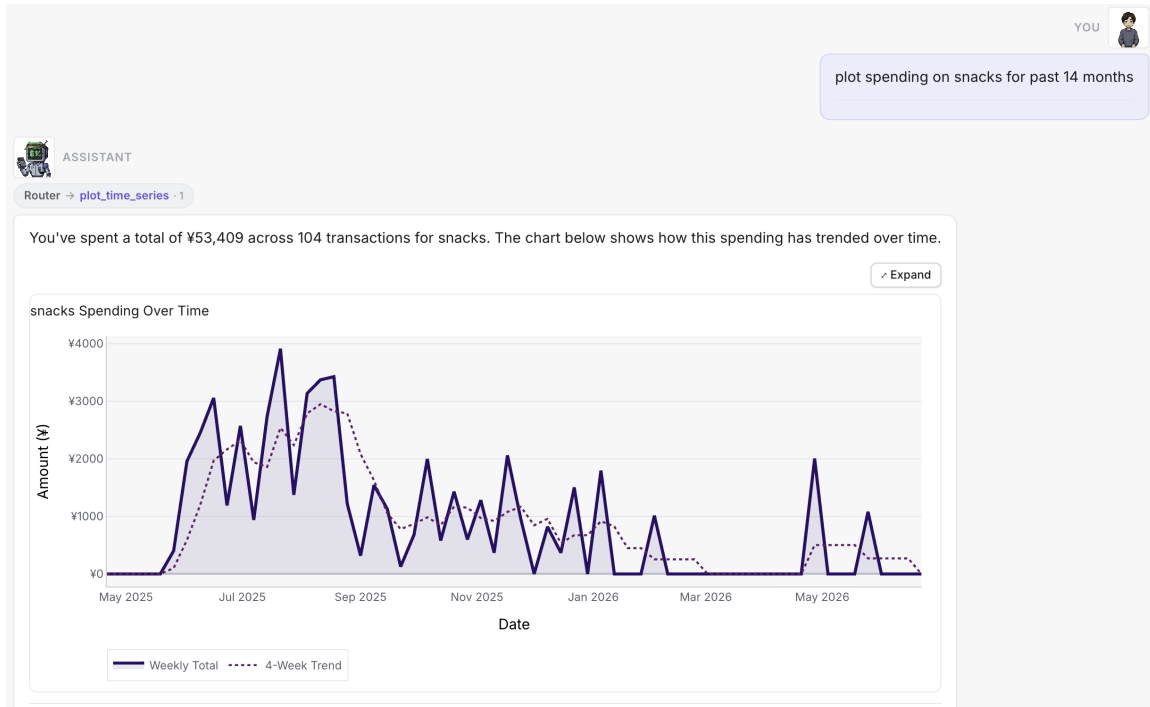


Figure 6: Real usage session: a user queries snack spending trends in informal language. The dual-agent Qwen3.5-0.8B pipeline correctly invokes `plot_time_series`, producing an actionable spending trend—entirely on-device with no data transmitted externally.

- We do not evaluate the *downstream correctness* of the financial analytics produced—only whether the tool call is correctly formed. Pro-  
duction deployment would require additional safeguards against calculation errors.
- Our privacy analysis is architectural (no data leaves the device) rather than cryptographic; we do not address model inversion or side-channel attacks on the local device itself.

## Ethics Statement

ExpenseSense is designed for on-device inference specifically to preserve user financial privacy. No real user financial data was used in the benchmark; all test cases reference synthetic expense categories. The system is intended to complement, not replace, professional financial advice. We recognise that even on-device models may produce incorrect financial summaries, and advocate for clear user-facing disclaimers in production deployments.

## Data and Code Availability

Code, prompts, benchmark data, and full results will be released on GitHub upon acceptance. Released artefacts include: all 115 test cases with ground-truth labels, the complete router and specialist prompt templates, inference configuration

## References

- Marah Abdin, Jyoti Aneja, Hany Awadalla, and 1 others. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.
- Yupeng Cao, Haohang Li, Weijin Liu, Wenbo Cao, Anke Xu, Lingfei Qian, and 1 others. 2026. FinTrace: Holistic trajectory-level evaluation of LLM tool calling for long-horizon financial tasks. *arXiv preprint arXiv:2604.10015*.
- Wei Chen and Zhiyuan Li. 2024. Octopus v2: On-device language model for super agent. *arXiv preprint arXiv:2404.01744*.
- Zhiyu Chen, Wenhui Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. *FinQA: A dataset of numerical reasoning over financial data*. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3697–3711.
- B. Efron. 1979. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26.

- Gemma Team. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*.
- Georgi Gerganov. 2023. [llama.cpp](#).
- Tom Gunter, Zirui Wang, Chong Wang, and 1 others. 2024. Apple intelligence foundation language models. *arXiv preprint arXiv:2407.21075*.
- Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, and 1 others. 2024. MiniCPM: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*.
- Ishan Kavathekar, Raghav Donakanti, Ponnurangam Kumaraguru, and Karthik Vaidhyanathan. 2025. [Small models, big tasks: An exploratory empirical study on small language models for function calling](#). In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, pages 1117–1126.
- LG AI Research. 2025. EXAONE 4.0: Unified large language models integrating non-reasoning and reasoning modes. *arXiv preprint arXiv:2507.11407*.
- Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, and 1 others. 2024. Personal LLM agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*.
- Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Jun Wang, and 1 others. 2024. Hammer: Robust function-calling for on-device language models via function masking. *arXiv preprint arXiv:2410.04587*.
- Liquid AI. 2026. LFM2.5 technical report. *Liquid AI Blog*.
- Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. 2024a. [MobileLLM: Optimizing sub-billion parameter language models for on-device use cases](#). In *Proceedings of the 41st International Conference on Machine Learning*, pages 32431–32454.
- Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, and 1 others. 2024b. APIGen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37.
- Helen Nissenbaum. 2004. Privacy as contextual integrity. *Washington Law Review*, 79(1):119–158.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. [The Berkeley function calling leaderboard \(BFCL\): From tool use to agentic evaluation of large language models](#). In *Proceedings of the 42nd International Conference on Machine Learning*, pages 48371–48392.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. [Gorilla: Large language model connected with massive APIs](#). In *Advances in Neural Information Processing Systems*, volume 37.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2024. [ToolLLM: Facilitating large language models to master 16000+ real-world APIs](#). In *Proceedings of the 12th International Conference on Learning Representations*.
- Qwen Team. 2026. Qwen3.5 technical report. *arXiv preprint arXiv:2604.15804*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36.
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. In *Advances in Neural Information Processing Systems*, volume 36.
- Yewei Song, Xunzhu Tang, Cedric Lothritz, Saad Ezzini, Jacques Klein, Tegawendé F. Bissyandé, Andrey Boytsov, Ulrick Ble, and Anne Goujon. 2025. [CallNavi: A challenge and empirical study on LLM function calling and routing](#). In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*.
- Robin Staab, Mark Vero, Mislav Balunović, and Martin Vechev. 2024. Beyond memorization: Violating privacy via inference with large language models. In *Proceedings of the 12th International Conference on Learning Representations*.
- Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, and Le Sun. 2023. ToolAlpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*.
- Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. 2024. On-device language models: A comprehensive review. *arXiv preprint arXiv:2409.00088*.
- Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. 2023. [FinGPT: Open-source financial large language models](#). In *International Symposium on Large Language Models for Financial Services (FinLLM 2023) at IJCAI 2023*.
- Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (LLM) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, 4(2):100211.

701 Jie Zhu, Yimin Tian, Boyang Li, Kehao Wu, Zhongzhi  
702 Liang, Junhui Li, Xianyin Zhang, Lifan Guo, Feng  
703 Chen, Yong Liu, and Chi Zhang. 2026. FinMCP-  
704 Bench: Benchmarking LLM agents for real-world  
705 financial tool use under the model context protocol.  
706 *arXiv preprint arXiv:2603.24943*.

## 707 A Complexity Scoring Features

| Feature            | Range | Description                                                 |
|--------------------|-------|-------------------------------------------------------------|
| $n_p$              | 0+    | Number of expected parameters                               |
| $d_{\text{date}}$  | 0–4   | Date complexity: none / year / month / range / specific day |
| $d_{\text{cat}}$   | 0–2   | Category distance: exact / alias / semantic                 |
| $d_{\text{rel}}$   | 0–1   | Relative time expression (“past X months”)                  |
| $d_{\text{multi}}$ | 0–2   | Compound parameter groups                                   |
| $d_{\text{abbr}}$  | 0–1   | Abbreviated year formats (“’24”)                            |
| $d_{\text{hol}}$   | 0–2   | Holiday/calendar knowledge required                         |

Table 7: Seven features of the complexity scoring function (Eq. 1).

## 708 B Sample Test Cases

| ID   | Query                                                          | Tool                 | Expected Params                                                                      | Tier |
|------|----------------------------------------------------------------|----------------------|--------------------------------------------------------------------------------------|------|
| TS01 | Can ya show spending on food for past 6 months?                | plot_time_series     | category=Food, months=6                                                              | L2   |
| DI06 | show breakdown of expenses for june 21 2024                    | plot_distribution    | year=2024, month=6, day=21                                                           | L2   |
| CP24 | compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026 | plot_comparison_bars | category=grocery, y1=2024, sm1=11, ey1=2025, em1=4, y2=2025, sm2=11, ey2=2026, em2=4 | L3   |
| CT07 | sum total spent on metro for july 2024?                        | calculate_total      | category=tokyo metro, year=2024, month=7                                             | L2   |
| TP06 | top 5 expenses past month? disregard rent                      | get_top_expenses     | n=5, months=1, ignore_rent=true                                                      | L2   |

Table 8: Sample test cases. Note informal language, abbreviations, and semantic mappings (“metro”→“tokyo metro”, “groceries”→“grocery”).

## 709 C Qualitative Error Examples

710 Table 9 illustrates representative failures for the dominant error types in Qwen3.5-2B (dual mode).

| Error Type        | Query                                                            | Expected         | Model Output                                                                |
|-------------------|------------------------------------------------------------------|------------------|-----------------------------------------------------------------------------|
| PARAM_MISSING     | “Show spending trend for the last 3 months”                      | {months: 3}      | { } — months omitted in the relative-time case                              |
| PARAM_VALUE_WRONG | “compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026” | {em1: 4, em2: 4} | {em1: 11, em2: 11} — confuses start month for end month in cross-year range |
| PARAM_EXTRA       | “How much did I spend on food?”                                  | {category: Food} | {category: Food, months: 1} — spurious time filter restricts scope          |

Table 9: Representative failure cases for Qwen3.5-2B (dual mode). Note that PARAM\_EXTRA is particularly dangerous in financial contexts: the spurious months=1 filter silently restricts “total food spending” to “last month’s food spending”.